INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Investigación y análisis previo sobre estructuras tipo Pila y Cola
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - "B"
Alumnos participantes:	Rodriguez Jimenez David Israel
Asignatura:	Estructura de Datos
Docente:	Ing. Caiza Caizabuano Jose Ruben

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:
Analizar situaciones problemáticas reales y computacionales en las que el orden de inserción y extracción de elementos determina la solución, con el fin de identificar la utilidad de las estructuras de datos tipo pila y cola antes de su implementación en lenguaje C++.

Específicos:

• Investigar los fundamentos conceptuales de las estructuras de datos tipo pila y cola.
• Diferenciar el comportamiento LIFO y FIFO mediante el análisis de casos prácticos.
• Representar secuencialmente la entrada y salida de elementos en problemas concretos.
• Formular soluciones preliminares utilizando diagramas, tablas de seguimiento y pseudocódigo.
• Desarrollar criterio analítico para seleccionar la estructura adecuada según el problema Planteado

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8
No presenciales: 0

2.4 Instrucciones

- El trabajo se desarrollará de manera individual.
- La redacción debe ser formal, clara y coherente, utilizando lenguaje técnico apropiado.
- No se aceptarán definiciones copiadas textualmente sin explicación propia del estudiante.
- Se valorará el análisis, la argumentación y la correcta representación de los casos de
- estudio.
- El producto final deberá entregarse en formato Word o PDF, según la indicación del
- docente.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial
- TAC Computadora con máquina virtual Linux (AlmaLinux, Ubuntu, Debian).
- Acceso a terminal Bash.
- Herramientas de texto (nano, vim, gedit).

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☐Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☐Inteligencia Artificial
Otros (Especifique): _____

2.6 Actividades por desarrollar

1. Investigación conceptual

Realice una revisión bibliográfica y explique con sus propias palabras los siguientes conceptos:

• Estructura de datos lineal
• Pila
• Cola
• Principio LIFO
• Principio FIFO
• Operación de inserción
• Operación de eliminación
• Implementación mediante arreglos
• Implementación mediante nodos

Para cada concepto principal, incluya un ejemplo sencillo que facilite su comprensión.

2. Análisis del caso 1: Paréntesis balanceados

Analice el problema de validación de expresiones con símbolos de apertura y cierre. Trabaje con las siguientes expresiones:

• (a+b) * (c-d)
• [(a+b) * {c/d}]
• (a+b]
• ((a+b)

Para cada expresión, desarrolle el seguimiento paso a paso del procesamiento de símbolos, identifique los elementos que ingresan y salen, explique por qué la expresión es válida o inválida, y determine qué tipo de estructura modela correctamente el problema.

3. Análisis del caso 2: Atención en ventanilla

Considere un sistema de atención al cliente en el que llegan las siguientes personas en este orden: Ana, Luis, Pedro, Marta y José.

• Se atienden los tres primeros clientes.
• Llega una nueva cliente llamada Carla.
• Luego se atienden dos clientes más.

Con base en esta situación, represente secuencialmente el orden de llegada y atención, elabore una tabla o esquema del estado de la estructura en cada paso, identifique los elementos que ingresan y salen, y justifique la estructura más adecuada.

4. Comparación analítica de los dos casos

Elabore un análisis comparativo entre ambos problemas respondiendo de forma argumentada a las siguientes preguntas:

• ¿Qué diferencias existen entre el procesamiento de los elementos en ambos casos?
• ¿En cuál de los casos el último elemento ingresado debe salir primero?
• ¿En cuál de los casos el primer elemento ingresado debe salir primero?
• ¿Qué errores se producirían si se utilizara una estructura incorrecta en cada caso?
• ¿Por qué el análisis del problema es indispensable antes de programar una solución?

5. Propuesta de solución lógica

Sin desarrollar todavía el programa en C++, redacte el pseudocódigo de solución para ambos

casos:
• Verificación de paréntesis balanceados.
• Simulación de atención en ventanilla.
El pseudocódigo debe considerar explícitamente operaciones como insertar, eliminar, verificar
si la estructura está vacía y consultar el siguiente elemento a procesar

2.7 Resultados obtenidos

1. Investigación conceptual

• Estructura de datos lineal

Una estructura de datos lineal es cuando los datos se alinean secuacioalmente, se puede decir que cada elemento tiempo un sucesor y un predecesor excepto el primero y el ultimo, los elementos se almacenan y accedes de manera ordenada uno tras otro. Las listas, pilas y colas son consideradas datos lineal en cambios lo rafos y arboles son no lineales

Es decir los datos lineales son aquellos que llevan un orden un elemento conece quien esta detrás y quien esta adelante y asi el que esta adelante conece al de atrás y al que esta delante, los datos se abtraen asi mismo no necesitan saber que elemento hay mas alla de su sucesor.

• Pila

Es un tipo de estructura donde se organiza en orden siguiendo el método LIFO **(Last In, First Out)** en este caso Pila funciona de tal forma que por ejemplo un tubo de papas fritas para llegar a la última papa debes sacar todas las anteriores para llegar a ella.

Una aplicación real: Historial de navegación, ya que cada página nueva se pone encima una de otra y cuando das clic atrás estas sacando o (Pop) de la página actual para ver la anterior

Operaciones fundamentales: Push (insertar en el tope) y Pop (eliminar del tope), junto con funciones auxiliares como PilaVacia, PilaLlena y Tope.

• Cola

Este es más fiel al método FIFO, el que entra primero sale primero.

Por ejemplo en nuestra carrera los sistemas operativos en específico la gestión de procesos funciona de forma similar ya que el procesador puede ser rápido pero solo hace una tarea a la vez pero al abrir muchas aplicaciones a la vez el sistema pone en cola de listos para darles un turno de ejecución

Un ejemplo real: Impresoras, su sistema busca un orden de salida a las impresiones seguidas el primero en ejecutarse será el primero en imprimirse

Operaciones: Insertar (agregar al final) y Quitar (retirar del frente), con auxiliares como ColaVacia, ColaLlena y Frente.

• Principio LIFO

LIFO (Last In, First Out) o en español el ultimo en estrar, el primero en salir. El metodo LIFO es el que mejor describe a la Pila, que se refiere a si misma como un monton de algo como ejemplo una pila de carpetas sobre un escritorio el oficinista para anlizar las carpetas primero analizara la ultima en ser apilada hasta llegar a la primera que fue apilada siendo la ultima de la pila.

• Principio FIFO

FIFO (First in, First on) que en español es el primero en entrar es el primero en salir, este principio es que mas caracteriza a la Cola ya que el primero es una cola sera el primero en salir un ejemplo es una fila del banco el primero en llagar al banco sera el primero en salir

• Operación de inserción

Agregar un nuevo elemento a la estructura de datos:

Pila: se llama “Push” y coloca elemento en el tope de la pila

Push(5) → la pila pasa de [3, 7] a [3, 7, 5]

Cola: se llama “Insertar” y coloca el elemento al final de la cola

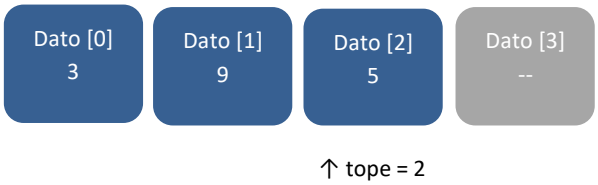
Insertar("Pedro") → la cola pasa de [Ana, Luis] a [Ana, Luis, Pedro].

Antes de insertar debe haber un condicionamiento para verificar que la estructura no esta llena

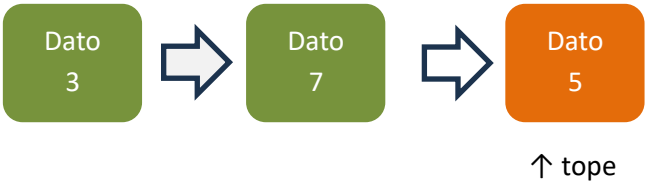
• Operación de eliminación

Pila: se llama “Pop” este quita el ultimo elemnto de la pila
Si la pila es [3, 7, 5], hacer Pop la deja como [3, 7]
Cola: se llama “Quitar” este quita el primer elemnto de la cola
Si la cola es [Ana, Luis, Pedro], hacer Quitar la deja como [Luis, Pedro]
Antes de eliminar un elemento de la estructura se debe verificar que no esta vacia

• Implementación mediante arreglos
Arreglos o tambien conocidos como estructuras listas estaticas que necesita una predefinicion de memoria al declarar la estructura.
Pila: Un arreglo Datos[MAX] y un entero Tope
Cola: un arreglo Datos[MAX], un entero Frente “enqueue” y un entero Final “dequeue”



• Implementación mediante nodos
En la implementacion mediante nodos cada elemento es un nodo y este contiene un dato y un puntero que apunta al siguiente nodo para solicitar la memoria con “malloc”, para liberar con “free”



3. Análisis del caso 1: Paréntesis balanceados

Analice el problema de validación de expresiones con símbolos de apertura y cierre. Trabaje con las siguientes expresiones:
• (a+b) * (c-d)
• [(a+b) * {c/d}]
• (a+b]
• ((a+b)

Para estos problemas el principio Pila es el mas adecuado para este problema porque el ultimo parentesis abiertos debe ser el primero en cerrarse.
La logia es por cada simbolo de apertura (.[,{, se hace push, por cada simbolo de cierre),],} verifica el tope de la pila es decir verifica si contiene el simbolo de apertura correspondiente y hace Pop, si la pila que vacia la expresion es valida

• (a+b) * (c-d)

Paso	**Símbolo**	**Operación**	**Pila**
1	(	Push (	[( ]
2	a+b	Ignorar	[( ]
3	)	Pop	[ ]
4	*	Ignorar	[ ]
5	(	Pop	[( ]
6	c-d		[( ]
7	)		[ ]

8	Fin	Pila vacia	[ ]

Valida porque la Pila queda vacia al finalizar

• [(a+b) * {c/d}]

Paso	Símbolo	Operación	Pila
1	[	Push [	[[ ]
2	(	Push (	[[, ( ]
3	a,+,b	Ignorar	[[, ( ]
4	)	Pop (→ coincide con)	[[ ]
5	*	Ignorar	[[ ]
6	{	Push {	[[, {]
7	c,/,d	Ignorar	[[, {]
8	}	Pop { → coincide con }	[[ ]
9	]	Pop [→ coincide con]	[ ]
10	Fin	Pila vacía →	[ ]

Valida porque la Pila queda con apertura (al finalizar

• (a+b]

Paso	Símbolo	Operación	Pila
1	(	Push (	[( ]
2	a,+,b	Ignorar	[( ]
3	]	Tope = (≠ [→ ERROR de coincidencia	[( ]

Invalida porque la Pila queda con apertura en el tope

• ((a+b)

Paso	Símbolo	Operación	Pila
1	(	Push (	[( ]
2	(	Push (	[(, ( ]
3	a,+,b	Ignorar	[(, ( ]
4	)	Pop (→ coincide con)	[( ]
5	Fin	Pila NO vacía → falta cerrar 1 paréntesis	[( ]

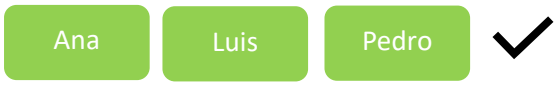
Invalida porque queda un (sin cerrar al finalizar

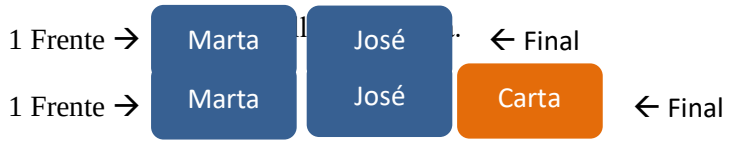
3. Análisis del caso 2: Atención en ventanilla

Considere un sistema de atención al cliente en el que llegan las siguientes personas en este orden: Ana, Luis, Pedro, Marta y José.

Frente = Ana
Final = José

• Se atienden los tres primeros clientes.
Se eliminan los 3 primeros (ya fuerona atendidos)





• Luego se atienden dos clientes más.
Se eliminan los 2 primeros (ya fuerona atendidos)

Tabla:

Cliente	Llegada	Posición en cola	Atención
Ana	1ª	Frente inicial	1ª atendida
Luis	2º	Posición 2	2º atendido
Pedro	3º	Posición 3	3º atendido
Marta	4ª	Posición 4	4ª atendida
José	5º	Posición 5	5º atendido
Carla	6ª (nueva)	Final (posición 3)	Pendiente de atención

La cola FIFO es la unica estrucutra de datos adecuados para este problema porque nos garantiza que el primer cliente sera el primero en salir es decir ser atendido siguiendo el orden de llagada. Si no cumplimos esto para este ejemplo seria un error ya que no atenderiamos al primero en llegar lo cual seria un error.

4. Comparación analítica de los dos casos

Elabore un análisis comparativo entre ambos problemas respondiendo de forma argumentada a las siguientes preguntas:

• ¿Qué diferencias existen entre el procesamiento de los elementos en ambos casos?
LIFO/Pila: El simbolo que da la apertura mas reciente debe ser el primero en verificarse al encontrar un cierre ocea es el orden inverso al de llegada (no es el primero)
FIFO/Cola: El orden de llegada de los clientes predomina el primero en llegar sera el primero en ser atendido

• ¿En cuál de los casos el último elemento ingresado debe salir primero?
LIFO/Pila: El ultimo elemento ingresado o en este caso el parentesis abierto mas recientemente debe cerrarse antes que los demas, quiere decir que el metodo Pila es el unico en el que el ultimo elemento sera el primero en salir

• ¿En cuál de los casos el primer elemento ingresado debe salir primero?
FIFO/Cola: El cliente que llega primero sera el primero en ser atendido, Cola es el unico que cumple esta regla de ser el primero en entrar y el primero en salir

• ¿Qué errores se producirían si se utilizara una estructura incorrecta en cada caso?
LIFO/Pila: si se usa Cola en para este caso lo que trataria de hacer al encontrar) se intentaria hacer pop del primer parentesis abierto, no del ultimo, se romperia dando error aunque sea una estructura valida

FIFO/Cola: Si se usa Pila para este caso lo que trataria de hacer es atender al ultimo cliente que entro lo que disgustaria a los demas clientes como Ana, luis y Pedro

• ¿Por qué el análisis del problema es indispensable antes de programar una solución?
Analizar el ejercicio es fundamental para determinar que tipo de estrucutra de datos lineal se debe aplicar permitiendo indentificar que principio rigue el problema (LIFO O FIFO) sino se eligue la estructura correcta esto produce resultados logicamente erroneos aunque el programa compile sin errores.

5. Propuesta de solución lógica
Sin desarrollar todavía el programa en C++, redacte el pseudocódigo de solución para ambos casos:
• Verificación de paréntesis balanceados.

Pseudocodigo:

```
Pila: Registro
  Datos[MAX]: Carácter
  Tope: Entero
Fin Registro

P: Pila
Expresion[100]: Cadena
I: Entero
Simbolo, TopeActual: Carácter
Valida: Booleano

Módulo Inicializar()
  P.Tope ← -1
Fin Módulo

Módulo Push(Simbolo: Carácter)
  Si (PilaLlena() = Falso) entonces
    P.Tope ← P.Tope + 1
    P.Datos[P.Tope] ← Simbolo
  Fin Si
Fin Módulo

Módulo Pop()
  Si (PilaVacia() = Falso) entonces
    P.Tope ← P.Tope - 1
  Fin Si
Fin Módulo

Función Tope(): Carácter
  Si (PilaVacia() = Falso) entonces
    Regresa P.Datos[P.Tope]
  Fin Si
Fin Función

Función PilaVacia(): Booleano
  Regresa (P.Tope = -1)
Fin Función

Función PilaLlena(): Booleano
  Regresa (P.Tope = MAX - 1)
Fin Función
```

```
Función EsApertura(S: Carácter): Booleano
  Regresa (S = '(' o S = '[' o S = '{')
Fin Función

Función Coincide(Apertura: Carácter, Cierre: Carácter): Booleano
  Regresa ((Apertura = '(' y Cierre = ')') o
           (Apertura = '[' y Cierre = ']') o
           (Apertura = '{' y Cierre = '}'))
Fin Función

Programa Principal
  Inicializar()
  Escribir("Ingrese la expresión: ")
  Leer(Expresion)
  Valida ← Verdadero
  I ← 0

  Mientras (I < Longitud(Expresion) y Valida = Verdadero)
    Simbolo ← Expresion[I]
    Si (EsApertura(Simbolo)) entonces
      Push(Simbolo)                    // Insertar apertura en la pila
    Sino Si (Simbolo = ')' o Simbolo = ']' o Simbolo = '}') entonces
      Si (PilaVacia()) entonces        // Verificar si la estructura está vacía
        Valida ← Falso                 // Cierre sin apertura correspondiente
      Sino
        TopeActual ← Tope()            // Consultar el siguiente elemento a procesar
        Si (Coincide(TopeActual, Simbolo)) entonces
          Pop()                        // Eliminar la apertura emparejada
        Sino
          Valida ← Falso               // No coincide el tipo de paréntesis
        Fin Si
      Fin Si
    Fin Si
    I ← I + 1
  Fin Mientras

  Si (Valida = Verdadero y PilaVacia()) entonces
    Escribir("Expresión VÁLIDA")
  Sino
    Escribir("Expresión INVÁLIDA")
  Fin Si
Fin Programa
```

• Simulación de atención en ventanilla.

Pseudocodigo:

```
Cola: Registro
  Datos[MAX]: Cadena[30]
  Frente: Entero
  Final: Entero
Fin Registro

C: Cola
Cliente[30]: Cadena
Op: Entero
```

```
Módulo InicializarCola()
  C.Frente ← -1
  C.Final ← -1
Fin Módulo

Módulo Insertar(Nombre: Cadena)
  Si (ColaLlena() = Falso) entonces
    C.Final ← C.Final + 1            // Insertar al final
    C.Datos[C.Final] ← Nombre
    Si (C.Frente = -1) entonces
      C.Frente ← 0
    Fin Si
  Sino
    Escribir("Cola llena, no se puede agregar cliente")
  Fin Si
Fin Módulo

Módulo Quitar()
  Si (ColaVacia() = Falso) entonces
    Escribir("Atendiendo a: ", Frente()) // Consultar el siguiente a procesar
    C.Frente ← C.Frente + 1            // Eliminar del frente
    Si (C.Frente > C.Final) entonces    // La cola quedó vacía
      C.Frente ← -1
      C.Final ← -1
    Fin Si
  Sino
    Escribir("No hay clientes en espera")
  Fin Si
Fin Módulo

Función Frente(): Cadena
  Si (ColaVacia() = Falso) entonces
    Regresa C.Datos[C.Frente]          // Consultar primer elemento sin eliminarlo
  Fin Si
Fin Función

Función ColaVacia(): Booleano
  Regresa (C.Frente = -1)
Fin Función

Función ColaLlena(): Booleano
  Regresa (C.Final = MAX - 1)
Fin Función

Programa Principal
  InicializarCola()

  // Llegan los 5 primeros clientes
  Insertar("Ana")
  Insertar("Luis")
  Insertar("Pedro")
  Insertar("Marta")
  Insertar("José")

  // Se atienden los 3 primeros
  Quitar()    // Ana
```

```
  Quitar()    // Luis
  Quitar()    // Pedro

  // Llega una nueva cliente
  Insertar("Carla")

  // Se atienden 2 clientes más
  Quitar()    // Marta
  Quitar()    // José

  // Estado final
  Si (ColaVacia() = Falso) entonces
    Escribir("Clientes aún en espera. Siguiente: ", Frente())
  Sino
    Escribir("No quedan clientes en espera")
  Fin Si
Fin Programa
```

4.1 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☒ Trabajo en equipo
☒ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☐ Adaptabilidad
☐ Responsabilidad

4.2 Conclusiones

- Se Investigo los fundamentos conceptuales de las estructuras de datos tipo pila y cola.
- Se diferencia el comportamiento LIFO y FIFO mediante el análisis de casos prácticos.
- Se represento secuencialmente la entrada y salida de elementos en problemas concretos.
- Se formulo soluciones preliminares utilizando diagramas, tablas de seguimiento y
- pseudocódigo.
- Se desarrollo criterio analítico para seleccionar la estructura adecuada según el problema planteado